DietView technical reference

Package companion — what DietView contains and why

Source: `vignettes/articles/technical-reference.Rmd`
`technical-reference.Rmd`

This is the **technical companion** to DietView. It is distinct from the journal article — which argues *why* the package matters — and from the auto-generated function reference. It explains, in one place, what the package

really contains: every exported function, the logic of the code, the data structures it passes around, and the design decisions behind them.

What DietView is

DietView turns a **plain long table of stomach-content records** — one row per prey record, with columns for the predator, the stomach, the prey, an optional weight, and any covariates — into interactive, taxonomically complete **sunburst** views of predator diet. The input table carries *no taxonomy*: DietView adds the seven flagship Linnaean ranks (kingdom to species) itself, from WoRMS or from a lineage table you supply. From that single input it produces per-taxon importance in two currencies, an explicit "NA" category for unidentified prey, covariate stratified views, a digestion identification-bias diagnostic, and three outputs (a Shiny explorer, a self-contained HTML dashboard, and plain plotly figures).

Design principles (the why behind the code)

One input contract, spec-driven. DietView never hard-codes column names. You describe the roles of your columns once, with `dv_spec()`, and every other function reads that spec. This is what lets the same code serve any survey.

Taxonomy on demand, provenance-logged, never silently guessed. Prey are resolved to WoRMS in a strict order — known AphiaID, exact name, then optionally fuzzy name, then unresolved — and every prey carries a provenance record. Fuzzy matches are flagged, and `dv_taxonomy_report()` surfaces exactly what needs a human eye. Unaccepted names are followed to their valid AphiaID; results are cached.

The explicit-NA philosophy. Where identification stops, DietView keeps the deeper ranks as `NA` and renders them as an explicit "NA" category (white). Two consequences: within-rank percentages still sum to 100 %, and identification gaps stay visible instead of being absorbed into a functional grouping.

Within-rank importance as a genuine partition. Importance is the share of each node within its own rank, so every rank sums to 100 % independently — the NA node included. Each ring of the sunburst is its own complete pie.

Two currencies. Every view can be expressed by occurrence (how often) or weight (how much biomass); they routinely disagree, and showing both is part of an honest diet description. A `count` mode sizes segments by distinct diet items.

The `branchvalues = "total"` invariant. Aggregation is done at the finest rank and rolled up, so a parent's value equals the sum of its children — exactly what plotly's sunburst requires. A unit test guards it.

Fixed orientation for comparability. Sunbursts use `sort = FALSE` and a fixed start angle, so a taxon sits in the same place across covariate cells and cells can be compared directly.

Self-contained, reproducible outputs. The HTML dashboard embeds images as base64 and can render a single self-contained file; taxonomy can be frozen into a lineage table for a fully offline run.

Architecture and data flow

```
raw long table  +  dv_spec()                -> dietview_spec
      |
      v
dv_prepare(data, spec, taxonomy=)           -> dietview_prep
   |   taxonomy: 'columns' | 'table' | 'worms' | 'auto'
      |
      v
```

```
dv_build_tree(prep$data, ranks, weight/count) -> dietview_tree  (within-rank %)
      |
      +-------------------+--------------------+
      v                   v                    v
dv_sunburst()      dv_rank_table()   dv_digestion_diagnostic()
dv_build_html()/dv_deploy()/run_dietview()
```

The eight stages: (1) declare & validate — `dv_spec`, `dv_validate`; (2) attach taxonomy — `dv_prepare`, `dv_taxonomy_worms`, `dv_taxonomy_report`; (3) build covariates — `dv_add_period`, `dv_size_bins`; (4) compute metrics — `dv_build_tree`, `dv_rank_table`; (5) draw figures — `dv_sunburst`, `dv_palette`; (6) digestion bias — `dv_digestion_diagnostic`; (7) species cards — `dv_species_card_assets`; (8) ship — `dv_build_html`, `dv_deploy`, `run_dietview`.

Data structures

`dietview_spec` (from `dv_spec()`): the role → column mapping plus display ranks — `predator`, `stomach`, `prey_name`/`prey_id`, `weight`, `count`, `digestion`, `covariates`, `ranks`, `predator_latin`.

`dietview_prep` (from `dv_prepare()`): `data` (long table with rank columns attached), `spec`, `ranks`, and `taxonomy_log` (NULL unless taxonomy = `"worms"`). Its `print()` summarises counts, ranks, covariates, and WoRMS match breakdown.

`dietview_tree` (from `dv_build_tree()`): one row per sunburst node — `id`, `label`, `parent`, `rank`, `value_item`, `value_occ`, `value_w`, `pct_occ`, `pct_w` (each rank's percentages sum to 100).

Function reference (summary)

The complete per-argument reference lives on the reference index. In brief:

- **`dv_spec()`** — declare column roles once; a defensive constructor that errors if no prey key is given.
- **`dv_validate()`** — check a table against a spec; hard errors on missing required columns, soft warnings on missing weight/digestion.
- **`dv_prepare()`** — validate + attach lineage via `columns` / `table` / `worms` / `auto`; normalise "NA"/"" to real `NA`.
- **`dv_taxonomy_worms()`** — resolve prey to WoRMS (id/exact/fuzzy/unresolved), follow unaccepted names to valid AphiaID, cache, log provenance.
- **`dv_taxonomy_report()`** — return fuzzy + unresolved prey to review, sorted by records affected; optional CSV.
- **`dv_add_period()`** / **`dv_size_bins()`** — derive period and length-bin covariates with auto labels.
- **`dv_build_tree()`** — the metrics core: roll leaves up rank by rank, guarantee the total invariant, compute within-rank shares in both currencies.
- **`dv_rank_table()`** — the sorted importance table for one rank, NA included.
- **`dv_sunburst()`** — render the multi-rank sunburst; fixed orientation, currency-dependent sizing and hover.
- **`dv_palette()`** — stable label → colour dictionary; NA forced white.
- **`dv_digestion_diagnostic()`** — finest-resolved-rank shares by digestion level (via a `max.col` trick), optionally crossed with a covariate.
- **`dv_species_card_assets()`** — resolve an ID photo + description as a base64 data URI; graceful offline fallback.
- **`dv_build_html()`** / **`dv_deploy()`** — assemble and render the self-contained dashboard (searchable cards + per-predator tabs, up to three bands).
- **`run_dietview()`** — the interactive Shiny explorer.
- **`print.dietview_prep()`** — compact console summary.

The metrics core, in detail

`dv_build_tree()` carries three quantities through the tree — `value_item` (distinct items), `value_occ` (occurrence), `value_w` (weight). Missing ranks are filled with the explicit "NA" string so they become real nodes. Records are summarised at the finest rank into leaves, then the function loops from the coarsest rank inward, grouping leaves by the prefix of ranks and summing:

```
lvl <- lapply(seq_along(ranks), function(i) {
  cols_i <- ranks[seq_len(i)]
  g <- summarise(group_by(leaves, across(all_of(cols_i))),
                 value_item = sum(value_item),
                 value_occ  = sum(occ),
                 value_w    = sum(w))
  g$id     <- do.call(paste, c(g[cols_i], sep = '|'))
  g$parent <- if (i == 1) '' else do.call(paste, c(g[cols_i[-i]], sep = '|'))
  g$rank   <- rank_labels[i]
  g
})
```

Because each level is a roll-up of the same leaves, a parent's value equals the sum of its children (the total invariant). Within each rank, the occurrence and weight totals become the denominator, so `pct_occ` and `pct_w` give each node's share within its rank.

The sunburst call

```
plot_ly(tree, ids = ~id, labels = ~label, parents = ~parent, values = size_vals,
        type = 'sunburst', branchvalues = 'total', sort = FALSE, rotation = rotation,
        text = ~hovertext, hovertemplate = '%{text}<extra></extra>',
        marker = list(colors = ~color))
```

`sort = FALSE` plus a fixed `rotation` keep the reading order and start angle identical across covariate cells — what makes side-by-side comparison meaningful.

The digestion trick

For each record, the finest resolved rank is found by multiplying a logical matrix of "rank is non-NA" by the rank position and taking `max.col(..., ties = 'last')`; records with no resolved rank become "Unresolved". Grouping by digestion level (and optionally a covariate) then gives record and weight shares within each level.

Internal helpers and constants

`.DV_RANKS` (default kingdom..species), `.DV_D3` (the d3 category-20 palette), `.dv_titlecase()` / `.dv_slugify()` (display and file-name formatting), `.dv_fill_na()` (the explicit-NA conversion), `.dv_empty_tree()` (typed empty tree), `.dv_pct_fmt()` (hover formatting), and the dashboard pieces `.dv_head_tags()`, `.dv_home()`, `.dv_render()`.

Bundled data and the input contract

The package ships an example base table (`inst/extdata/dietview_example.csv`), a matching lineage lookup (`dietview_example_lineage.csv`), and a data dictionary. One row = one prey record in one stomach; free column names mapped via `dv_spec()`. The example deliberately includes class-only, order/family-only, and fully unidentified prey so the explicit-NA behaviour is visible end to end.

Dependencies

`dplyr` / `tidyr` (aggregation), `plotly` (sunburst), `htmltools` (dashboard HTML), `jsonlite` (Wikipedia/Commons), `knitr` (`image_uri`, vignettes), `grDevices` / `stats` / `tools` / `utils` / `rlang` (colours, stats, paths, tidy-eval). Suggested: `worrms` (WoRMS), `shiny` + `DT` (app), `rmarkdown` (self-contained render), `testthat` (tests).

Testing

The testthat suite locks the invariants that make the sunburst coherent: within-rank shares sum to 100 (NA included); the explicit “NA” node exists; the `branchvalues = 'total'` invariant holds; empty input yields a typed empty tree, not an error.

Extending DietView

Any survey works once it is shaped to one-row-per-prey-record and mapped with `dv_spec()`. Freeze taxonomy into a lineage table for a fully offline run; pass custom `ranks` or `colors`; add covariates and derive periods/size bins with the helpers; supply a lineage table for non-marine prey. See the Adapting DietView article.

On this page

Developed by Sosthène A. V. Akia, CADI Gulf Center.

Site built with pkgdown 2.2.1.